Republic of the Philippines

BICOL UNIVERSITY POLANGUI

COMPUTER STUDIES DEPARTMENT

Polangui, Albay

IT 112 - WEB SYSTEMS AND TECHNOLOGIES

PHASE 3
FRONTEND DEVELOPMENT BASED ON PROJECT DESIGN

BU STUDENT ACADEMIC TASK AND DEADLINE MANAGEMENT SYSTEM

Prepared by

Bea Roshan O. Pabilona

Eleanor Aimee O. Reantaso

Christian A. Balauro

Simranjit S. Singh

Karl James Matias

Group 5
BSIS-2B

Submitted to

GUILLERMO V. RED, JR.
Instructor

Project Design and Frontend Implementation

The Phase 3 frontend follows the use case diagram, MongoDB-style ERD, and system architecture in the Group 5 Phase 1 proposal (pages 3-7). The interface uses HTML, CSS, Bootstrap 5, and JavaScript.

Student

Students can open the registration and login forms, view task progress and deadlines, search and filter their personal tasks, and open task and subject forms. Task forms include the status, priority, due date, and optional subject specified in Phase 1.

Administrator

The administration page displays user accounts and tasks from multiple sample students. Account edit and delete dialogs support the account-management use case.

Navigation

The home page links to registration, login, and the two role previews. Student pages share links to Dashboard, Tasks, and Profile. The administration page groups account management and task oversight.

Frontend scope

Search and filtering work with fictional sample records. Form submissions display a message explaining which action requires an API connection. The future Express backend will handle authentication, validation, roles, task ownership, and database operations through Mongoose and MongoDB Atlas.

Bootstrap components

The pages use responsive containers, rows, and columns, a collapsible navbar, cards, form controls, tables, and modal dialogs. The screenshots below show the implemented pages and their relationship to the planned features.

Home

Page file: frontend/index.html

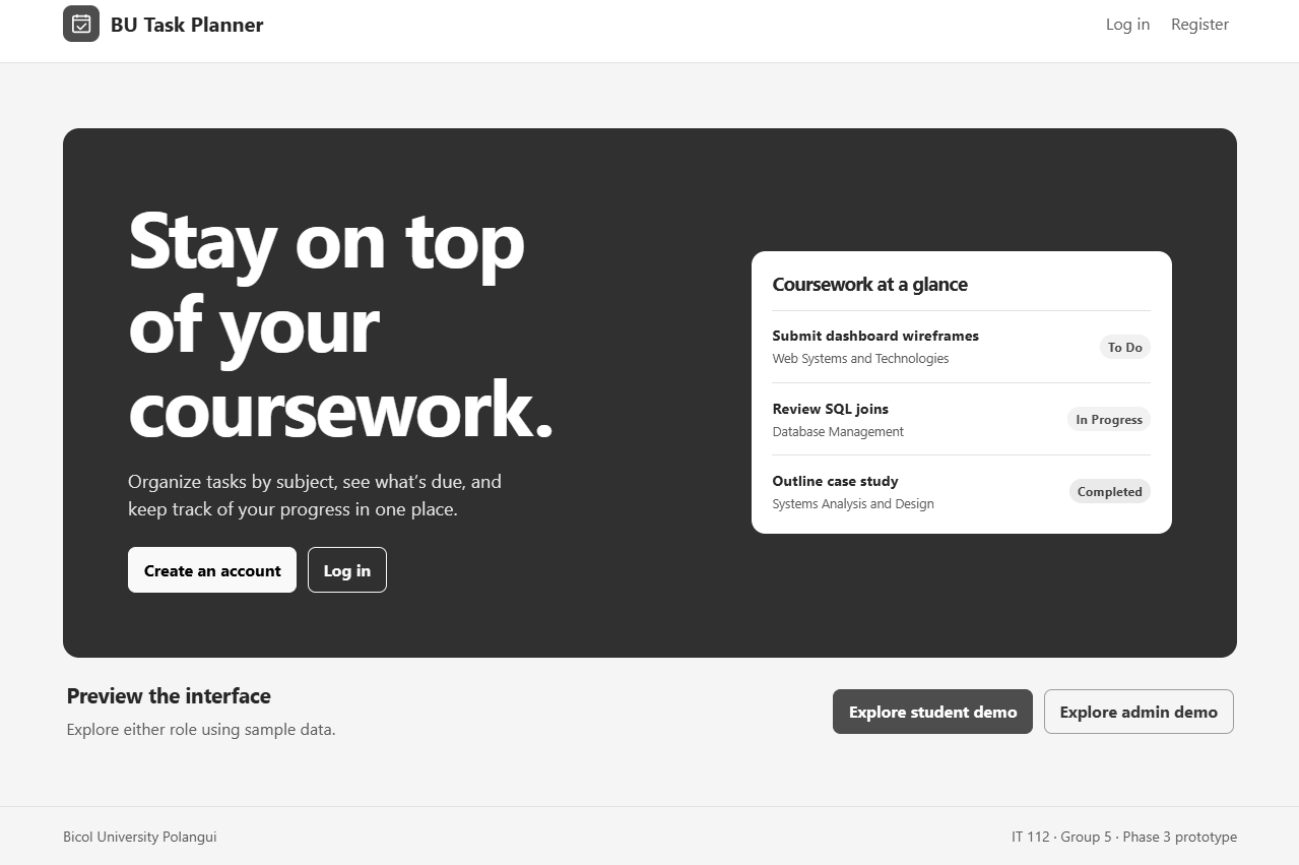


Figure 1. Home page.

Feature covered

Public entry point. Registration and login links lead to the account forms. The demo links open the student dashboard or administration page.

Log in

Page file: frontend/login.html

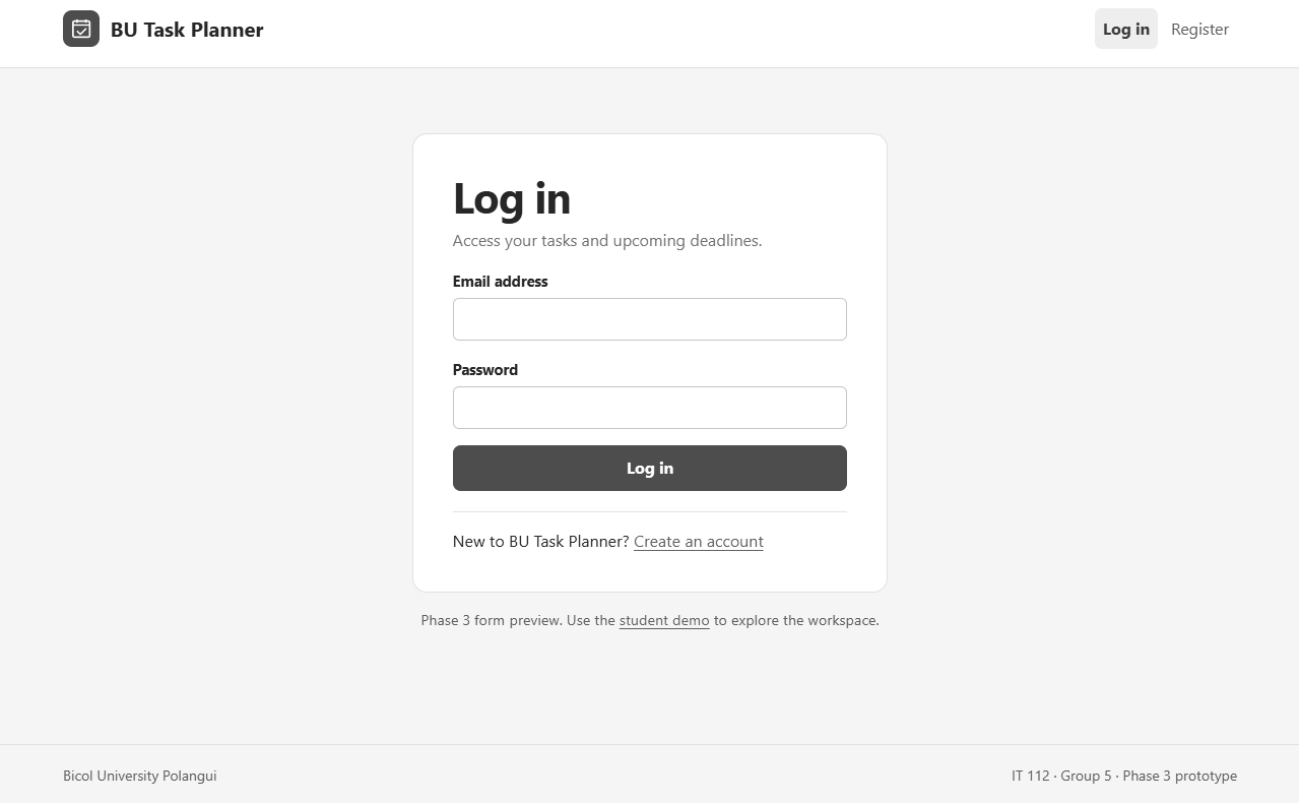


Figure 2. Log in page.

Feature covered

Register / log in. The form collects email and password. A future login endpoint will verify the password against Users.passwordHash and identify the role.

Register

Page file: frontend/register.html

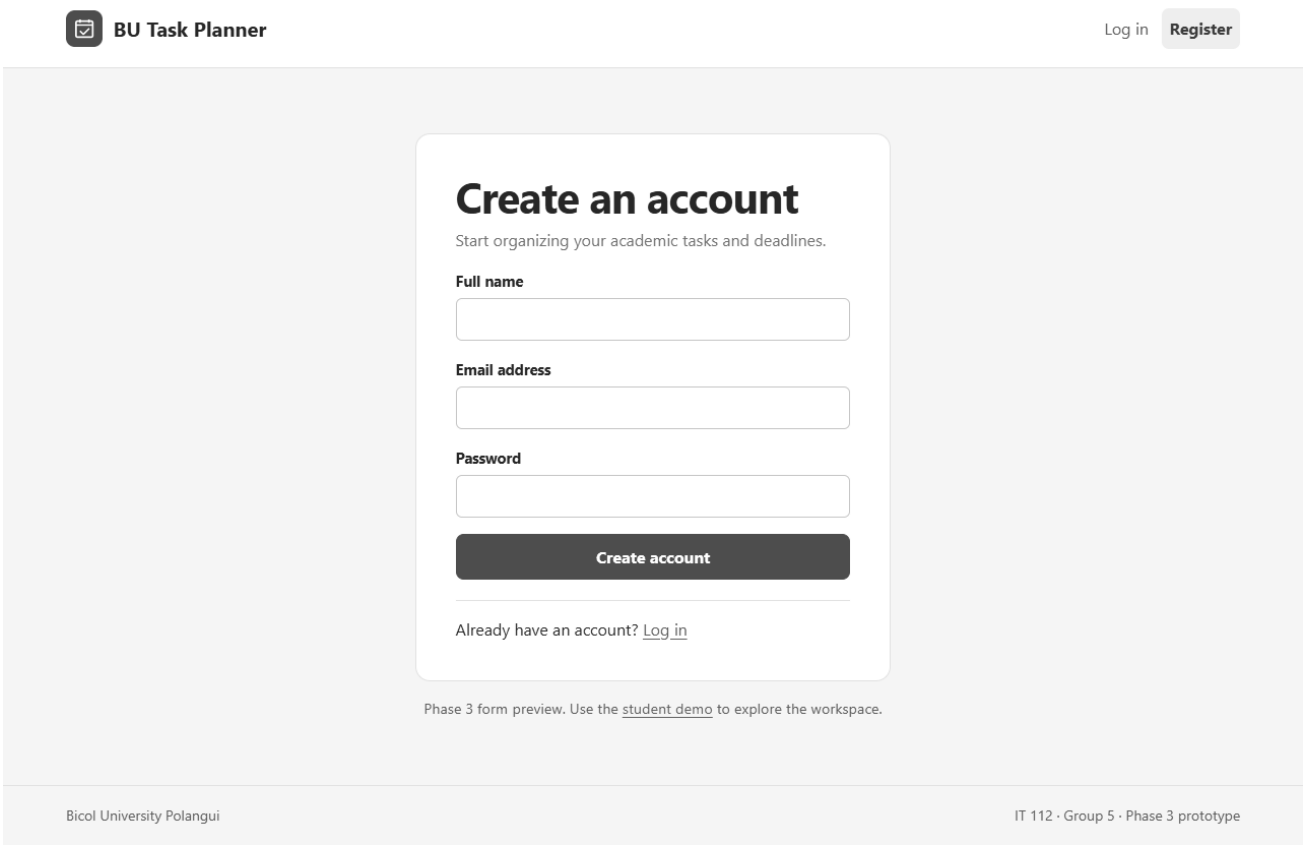


Figure 3. Register page.

Feature covered

Register / log in. A student enters name, email, and password. The future API will assign the user role and manage hashing and account metadata.

Student dashboard

Page file: frontend/dashboard.html

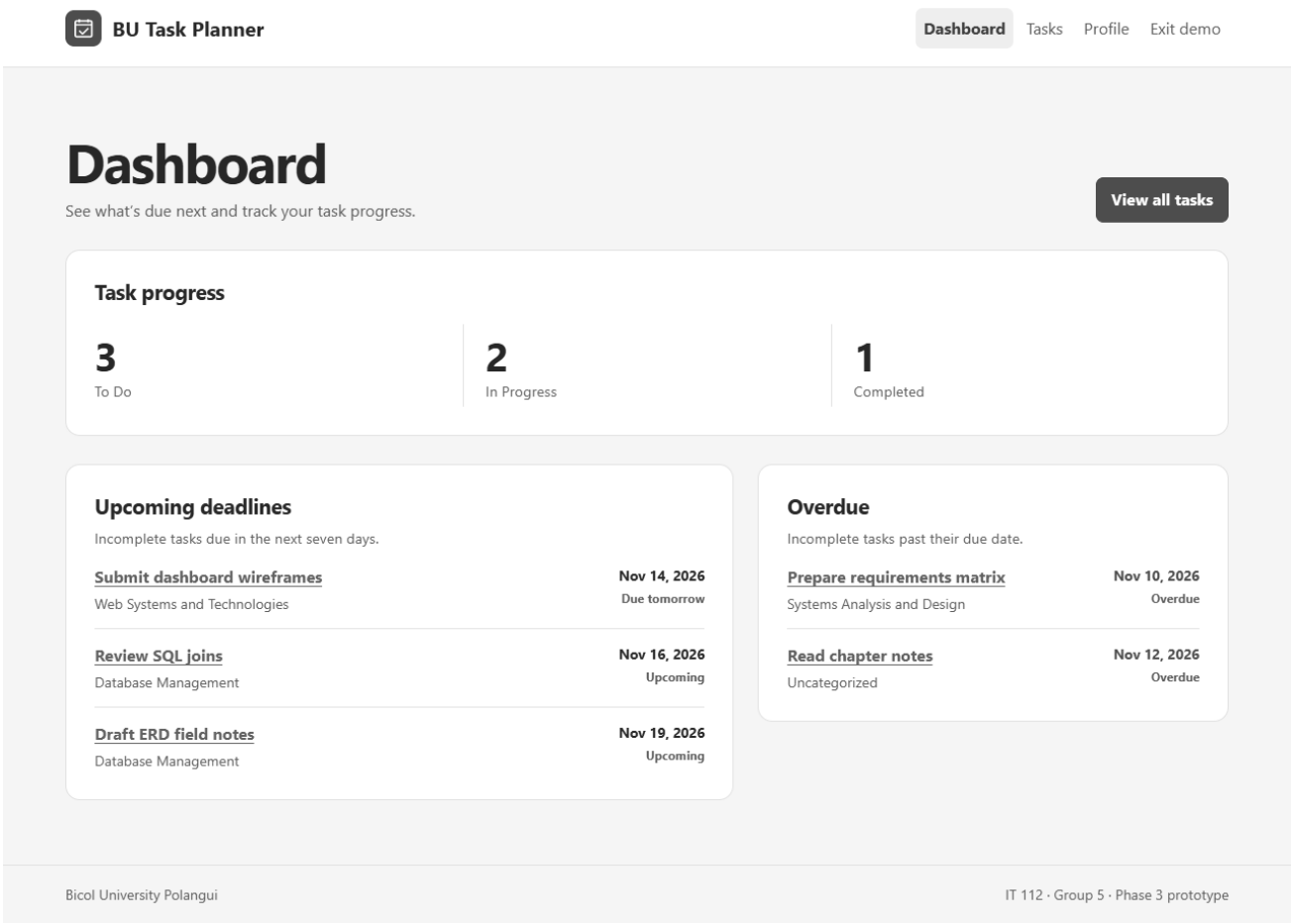


Figure 4. Student dashboard page.

Feature covered

View dashboard and deadlines. Status totals use Tasks.status. The deadline cards use Tasks.dueDate to show incomplete tasks due within seven days or already overdue.

Tasks

Page file: frontend/tasks.html

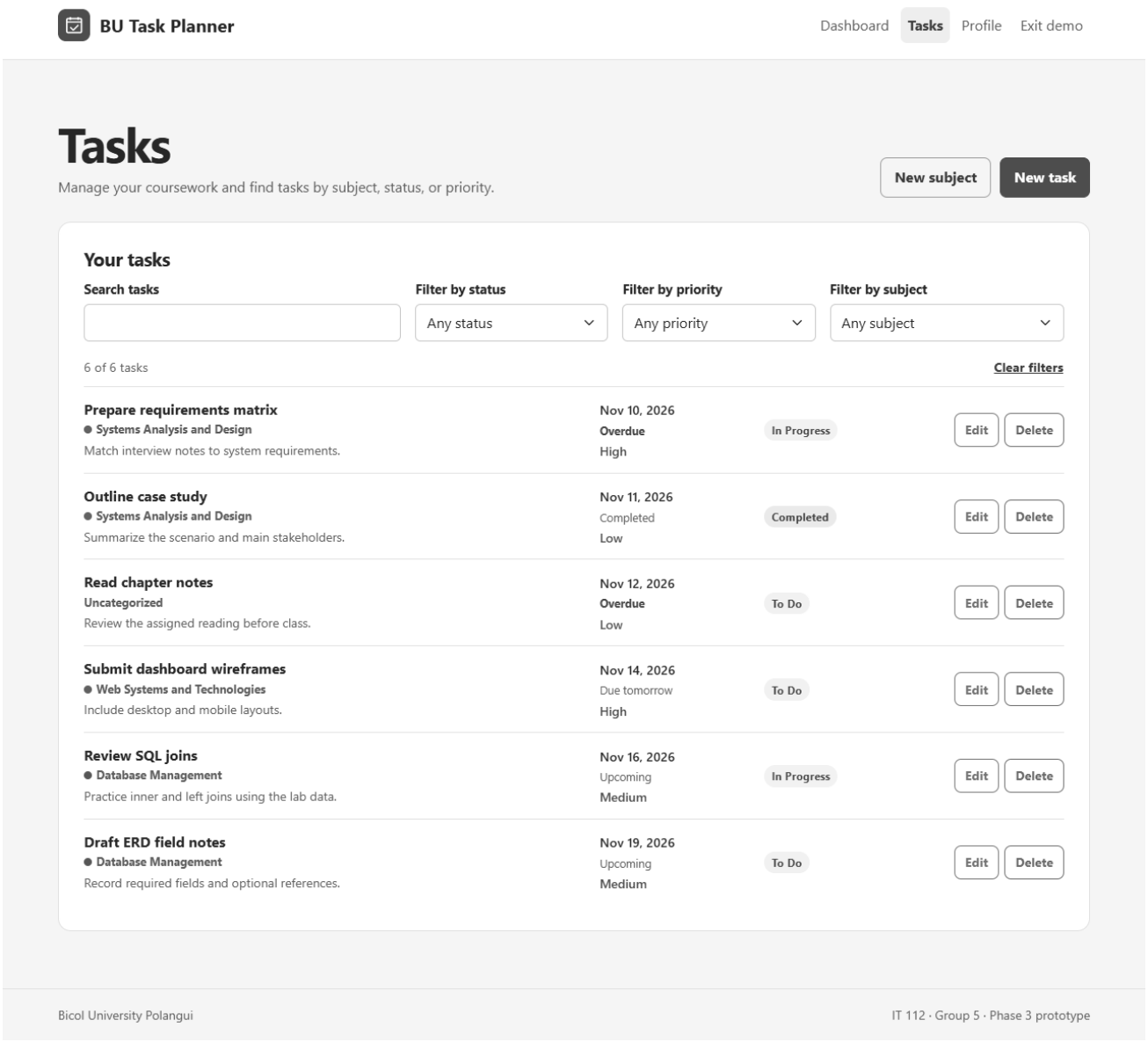


Figure 5. Tasks page.

Feature covered

Manage, search, and filter personal tasks. Task rows show title, description, subject, due date, priority, and status. Keyword, status, priority, and subject filters work on the sample records.

Profile

Page file: frontend/profile.html

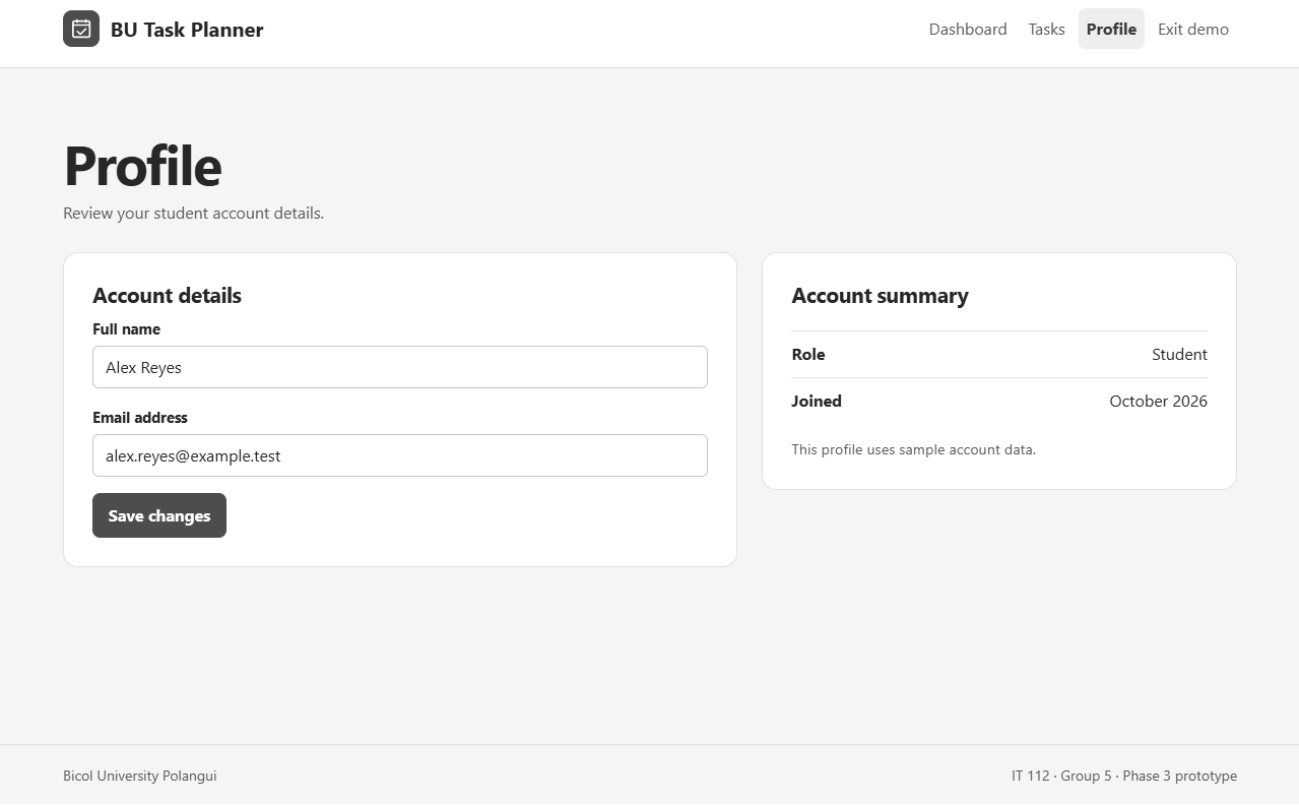


Figure 6. Profile page.

Feature covered

Review the student account. Name and email map to Users.name and Users.email. The account summary displays role and join date as read-only information.

Administration

Page file: frontend/admin.html

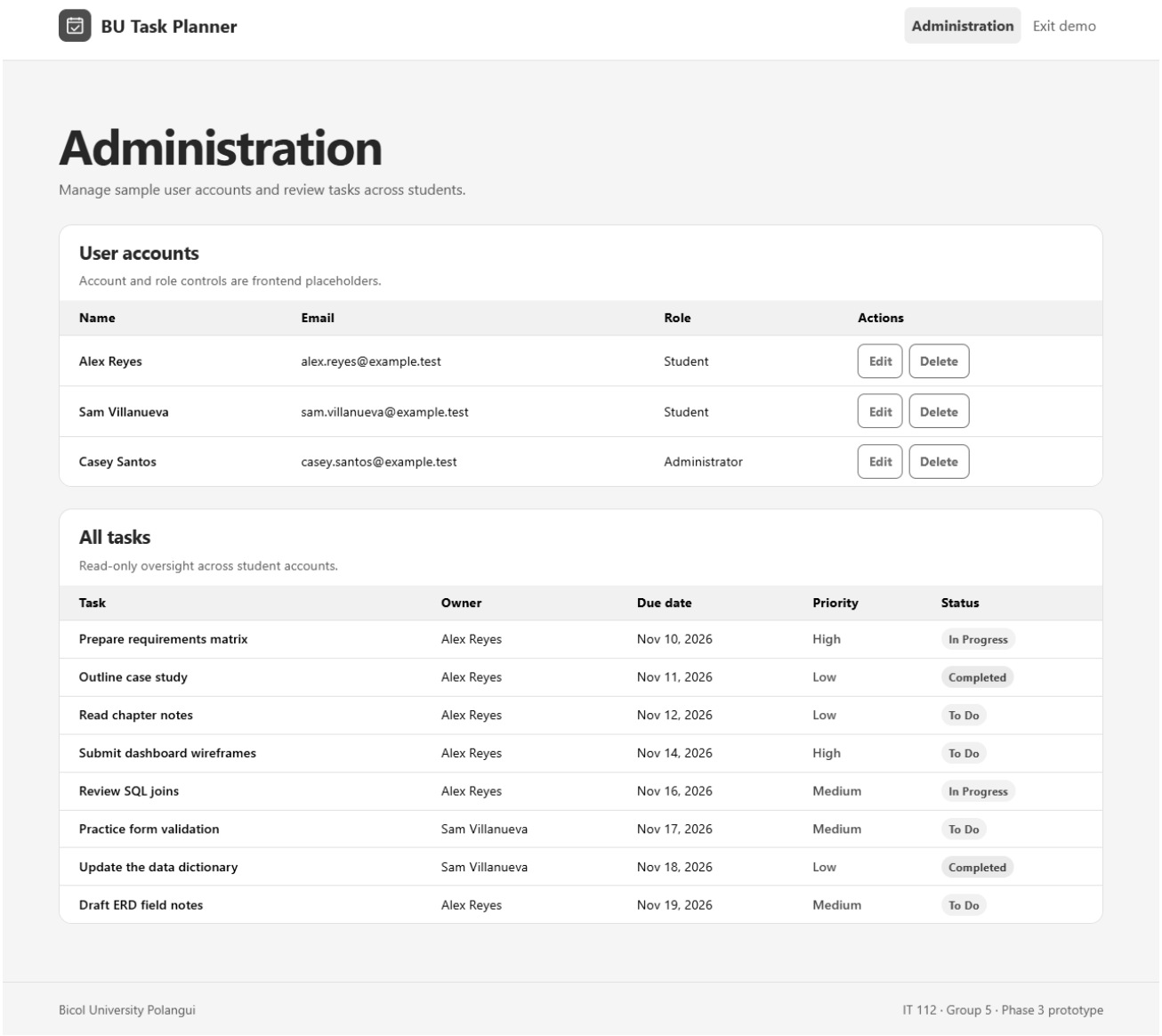


Figure 7. Administration page.

Feature covered

Manage accounts and oversee all tasks. The account table shows name, email, and role. Edit and delete dialogs carry the record ID. The oversight table shows sample tasks from multiple student owners.

ERD Field Mapping

Users

_id identifies the selected account in admin dialogs. **name** and **email** appear in the account forms. **passwordHash** will be produced by the backend from the password input. **role** uses user or admin. **createdAt** and **updatedAt** are server-managed timestamps.

Tasks

_id identifies the selected task for editing or deletion. **title**, **description**, **dueDate**, **priority**, and **status** appear in the task form. **category** stores an optional reference to Categories._id. **owner** will come from the authenticated user. **createdAt** and **updatedAt** are server-managed.

Categories

name and optional **color** appear in the subject form. **_id** and **createdAt** are server-managed. **owner** will come from the authenticated user. A task can remain uncategorized.

Allowed values

Task status uses To Do, In Progress, or Completed. Priority uses Low, Medium, or High. Student registration will receive the user role during future API integration.

Form structure and validation

Every form has an ID, action="#" and a GET or POST method. Inputs have names and IDs. Required fields use native browser validation. Whitespace-only task titles and names are rejected. Submissions display an action-specific placeholder message and move keyboard focus to it.

Task and Subject Forms

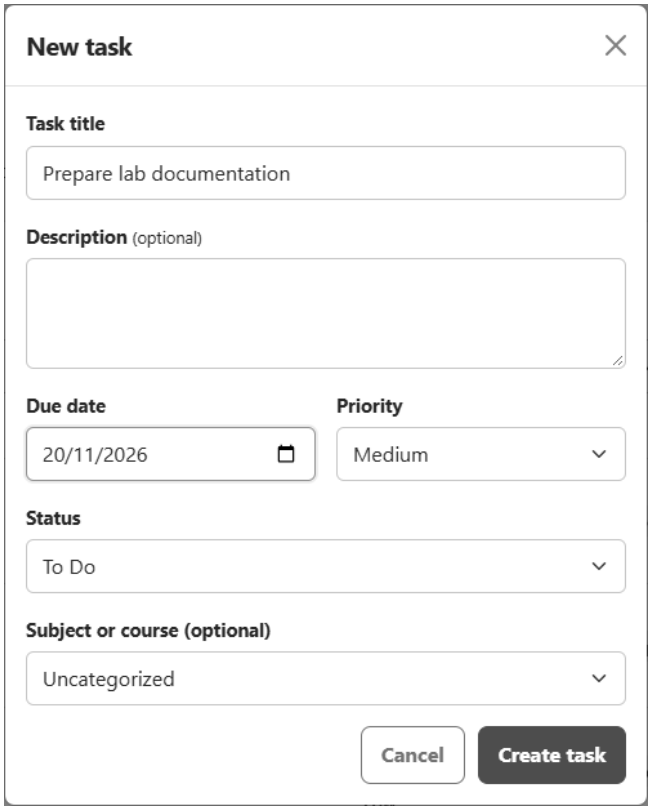


Figure 8. The task form collects a title, description, due date, priority, status, and optional subject. Title and due date are required.

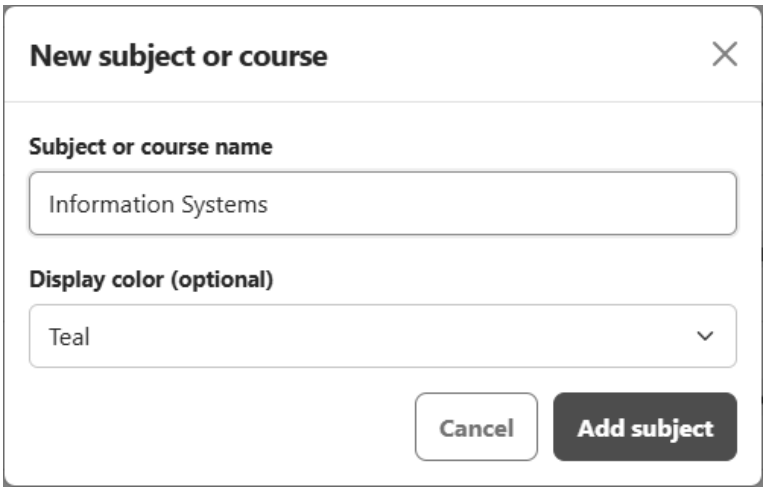


Figure 9. The subject form maps its name and optional display color to the Categories collection.

Account Management Forms

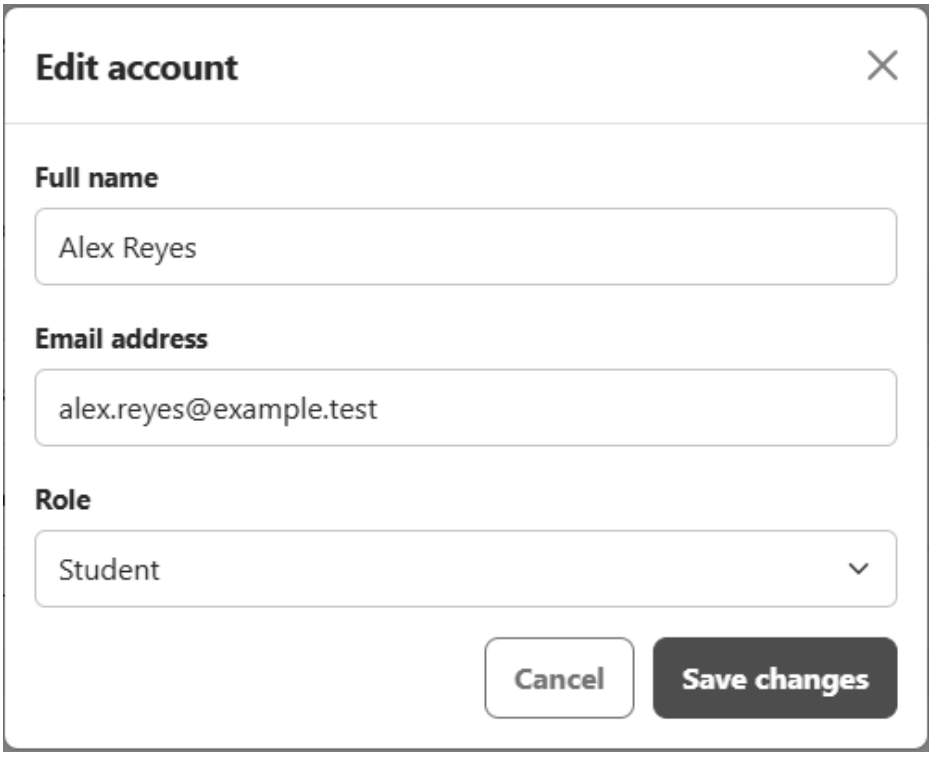


Figure 10. The account edit dialog collects name, email, and role. The selected account ID is carried in a hidden input.

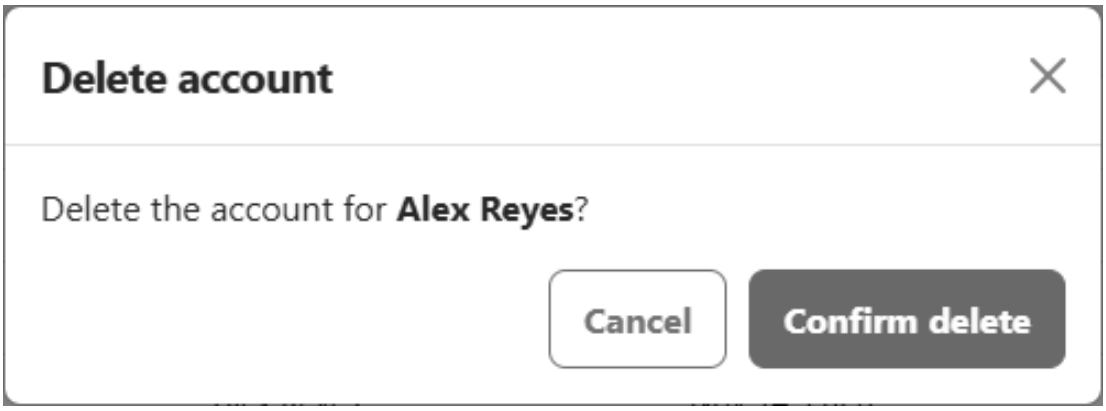


Figure 11. The delete dialog identifies the selected account before the placeholder action is submitted.

Task Editing and Deletion

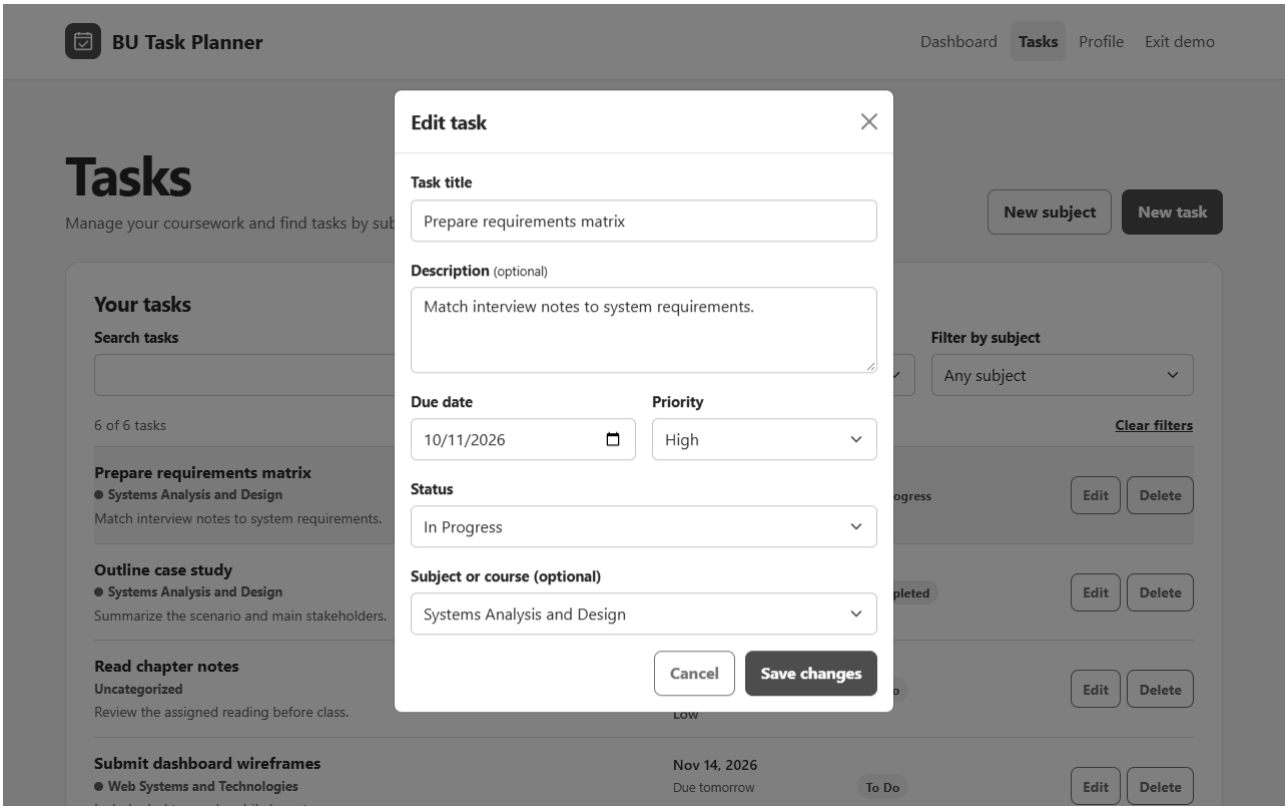


Figure 12. The edit dialog preloads the selected task and uses the same ERD fields as the create form.

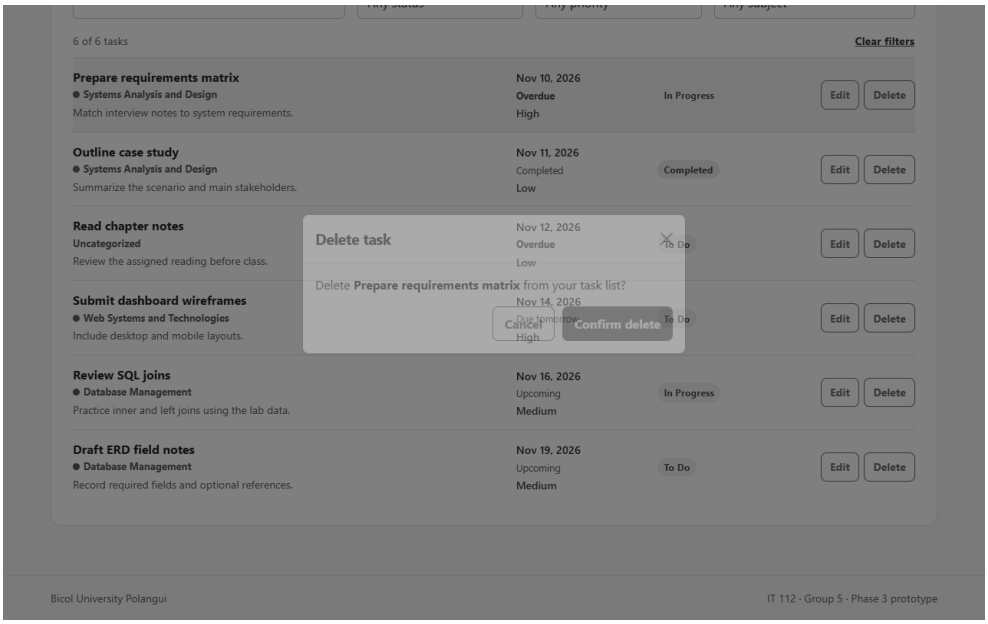


Figure 13. The task deletion dialog identifies the selected task. Submitting displays the API placeholder message.

Responsive Layout

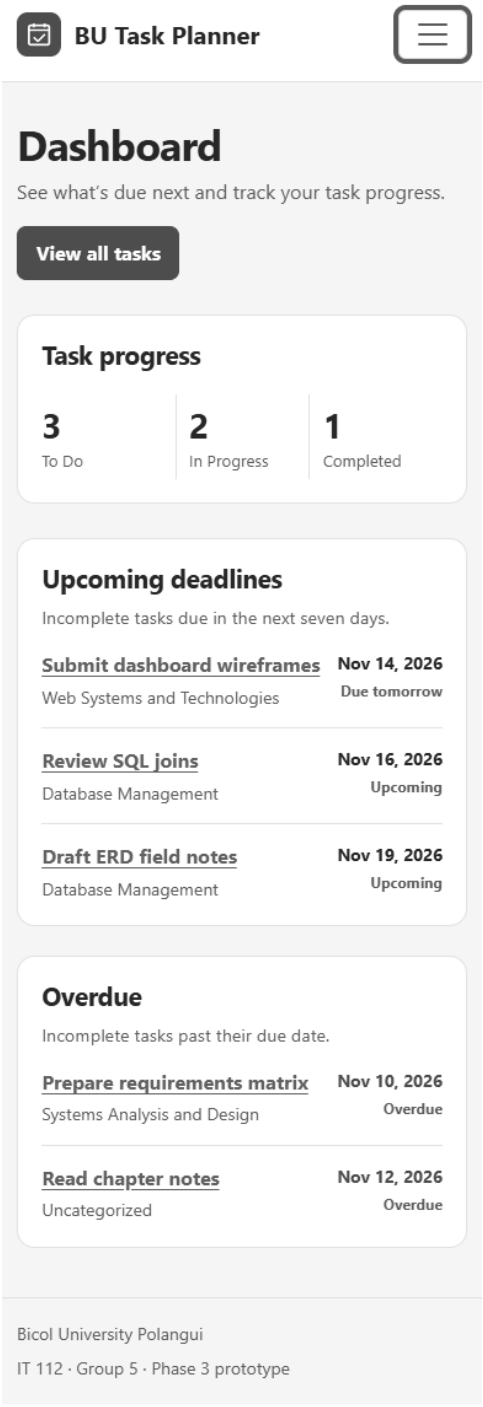


Figure 14. At a phone width, the dashboard cards stack and the navigation collapses into a menu button.

All seven pages were checked at desktop and phone widths. The browser checks covered navigation, search and filters, empty results, required fields, whitespace validation, dialog values, and placeholder feedback.

GitHub Folder Structure and Commits

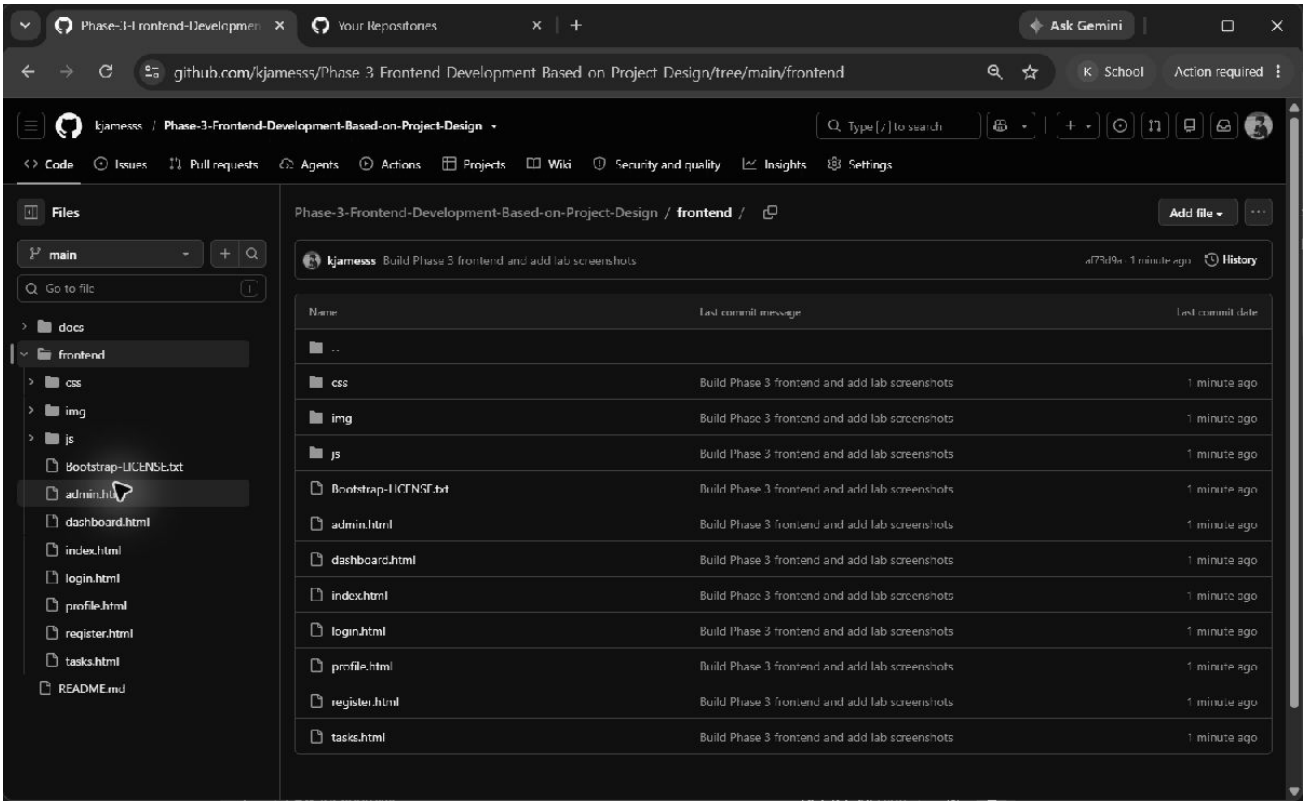


Figure 15. The GitHub frontend folder contains the seven HTML pages and the css, js, and img directories. The docs/lab11 folder contains the screenshots and PDF report.

Repository

kjamesss / Phase-3-Frontend-Development-Based-on-Project-Design

Recorded commits

42d35aa - Clarify deadlines and validate form previews

fab6d2a - Improve task scanning and keyboard focus

5f3c9c8 - Document field ownership and frontend checks

4a2d79f - Refine report and expand interface evidence

Individual Reflections

Bea Roshan O. Pabilona

Project Manager and Documentation Officer

I learned to turn the Phase 1 diagrams into a checklist for the frontend. The checklist made it easier to connect each screenshot to a student or administrator use case. In the next phase, I would use the same checklist to track which actions are ready for API testing.

Eleanor Aimee O. Reantaso

Front-End Developer / Front-End Manager

This phase showed me how a shared navbar and consistent form layout make separate pages feel like one application. I saw how much room the task filters and action buttons need when the layout becomes narrow. I would carry the same layout into the next phase and add feedback for API errors.

Christian A. Balauro

Backend Developer / Back End-Manager

The form previews helped me see what the Express routes will need to accept. The edit and delete dialogs made it clear that the server will need a record ID and an ownership or role check. For integration, I would start with authentication and then test what happens when a user tries to access a task they do not own.

Simranjit S. Singh

Database Manager

Mapping the screens to the ERD helped me see which values users supply and which values the database layer manages. The optional subject field is a good example because a task must still be valid without a category reference. In the next phase, I would verify that the allowed status and priority values match the Mongoose schema.

Karl James Matias

GitHub Manager

A consistent folder structure makes the frontend, screenshots, and report easier to find in GitHub. I learned why focused commit messages make it easier to trace the code and documentation changes. For the next phase, I would review the branch history before merging integration work.